- *Automatic migrations*—If you deploy your app to production automatically as part of some sort of DevOps pipeline, you'll inevitably need some way to apply migrations to a database automatically. You can tackle this situation in several ways, such as scripting the .NET tool, applying migrations in your app's startup code, using EF Core bundles, or using a custom tool. Each approach has its pros and cons.
- *Multiple web hosts*—One specific consideration is whether you have multiple web servers hosting your app, all pointing to the same database. If so, applying migrations in your app's startup code becomes harder, as you must ensure that only one app can migrate the database at a time.
- *Making backward-compatible schema changes*—A corollary of the multiple-web-host approach is that you'll often be in a situation in which your app accesses a database that has a newer schema than the app thinks. Normally, you should endeavor to make schema changes backward-compatible wherever possible.
- *Storing migrations in a different assembly*—In this chapter I included all my logic in a single project, but in larger apps, data access is often in a different project from the web app. For apps with this structure, you must use slightly different commands when using .NET CLI or PowerShell cmdlets.
- *Seeding data*—When you first create a database, you often want it to have some initial seed data, such as a default user. EF 6.x had a mechanism for seeding data built in, whereas EF Core requires you to seed your database explicitly yourself.

How you choose to handle each of these problems depends on the infrastructure and the deployment approach you take with your app. None is